

Introduction

Every Python developer hits this wall eventually: you write something useful, you send the `.py` file to a friend or coworker, and it just doesn't run on their machine. Maybe it's `ModuleNotFoundError`. Maybe it's a hardcoded import path that only resolves on your filesystem. Either way, the problem isn't their setup — it's that you never actually packaged your code, you just moved a file.

This matters far beyond "sharing a script with a friend." Every library you `pip install` — requests, numpy, flask — got to your machine through exactly the mechanism this lesson covers. Understanding packaging means understanding how the entire Python ecosystem distributes code, and it's a skill that separates someone who writes scripts from someone who ships software. It also comes up in real engineering work constantly: internal tools, shared utilities, anything more than one person needs to run.

By the end of this lesson you'll understand why `pip install` works at all, and you'll have built and published a package of your own.

Problem Overview

Here's the situation in plain terms: you have a folder of `.py` files that work fine when you run them yourself. You zip them up, or email one file, or push a folder to GitHub, and hand it to someone else. It doesn't run.

The reason is that your local success was an accident of context. Your imports resolve because you're standing in the right directory. Your dependencies are installed because you installed them at some point and forgot. None of that context travels with the file. `pip` — the tool responsible for installing Python code — has nothing to work with, because a folder of loose `.py` files carries no name, no version, and no dependency list. There's no contract for it to read.

Packaging is the act of writing that contract down, in a format every tool agrees on, so installation becomes mechanical instead of guesswork.

Example

Picture two versions of the same tiny library — a couple of functions for formatting text.

Version A: a folder called `textutils` with two `.py` files inside, pushed to GitHub. To use it, someone has to clone the repo, `cd` into it, and hope their working directory lines up with your imports. If your code imports `dateutil` and they don't have it installed, they find out at runtime, mid-function, with a traceback.

Version B: the same code, but with a `pyproject.toml` declaring the name `textutils`, version `0.1.0`, and `dateutil>=2.8` as a dependency. Someone runs:

```
pip install textutils
```

`pip` reads the file, sees the dependency list, installs `dateutil` first, then installs your code into the right place on their system. No cloning, no working-directory guesswork, no missing-package surprise.

Same code. The difference is entirely the presence of that one file and what it declares.

Intuition

Think about how you'd solve this if no tooling existed at all. You'd need to answer three questions before anyone else could run your code: what is this thing called, what does it need to run, and where does the code actually live relative to everything else in the project.

An experienced engineer doesn't start by picking a build tool — they start by asking what breaks first. And what breaks first is almost always dependencies: your code silently assumes libraries are present because you installed them once, months ago, and forgot. The second thing that breaks is structure: your imports work because your test scripts sit right next to your source files, and that adjacency is invisible until it's gone.

So the intuition is: packaging is just making those two invisible assumptions — "what this needs" and "where this lives" — explicit and machine-readable. Once they're explicit, any tool, on any machine, can reconstruct your working setup from scratch. That's the whole idea. Everything else — `pyproject.toml`, `src` layout, wheels, TestPyPI — is just the concrete mechanism for writing that down and verifying it before you commit to it publicly.

Brute Force Solution

The "brute force" version of packaging is what most people do without realizing it's a workaround: `setup.py`, plus a separate `requirements.txt`, plus README instructions telling people to `pip install -r requirements.txt` and hope the imports line up.

Idea: cobble together the contract using multiple files that different tools read differently — `setup.py` for build metadata (often with arbitrary executable code in it), `requirements.txt` for dependencies, and tribal knowledge for everything else.

Algorithm: 1. Write a `setup.py` with a `setup()` call describing the package. 2. Maintain a separate `requirements.txt` by hand. 3. Tell users to run both `pip install -r requirements.txt` and `pip install .` or similar. 4. Hope the two files stay in sync as the project evolves.

Advantages: it's the historical standard, and there's a lot of legacy tooling built around it.

Disadvantages: `setup.py` runs arbitrary Python during install — slower and a real security concern, since installing a package means executing code someone else wrote. The dependency list lives in two places that can drift apart. There's no single, static, tool-agnostic source of truth.

Complexity: not a runtime complexity question, but a maintenance one — every additional file describing your project is another place for the description to go stale. Time cost is mostly at install time (running setup code), and "space cost" is really cognitive: more files to keep consistent.

Optimal Solution

`pyproject.toml` replaces all of that fragmentation with one static, declarative file. It's not executed — it's read. That alone removes the security and consistency problems of `setup.py`.

The core contract, under the `[project]` section, has three parts that matter most:

Field	What it answers
<code>name</code>	What does someone type after <code>pip install</code> ?
<code>version</code>	Which release is this?
<code>dependencies</code>	What has to be installed before this code runs?

That's the whole thing any install tool needs to read.

Layout matters just as much as the file. The standard structure moves your actual package code into a `src/` folder, separate from tests and project files sitting at the top level:

```
my-project/
├── pyproject.toml
├── src/
│   └── mypackage/
│       ├── __init__.py
│       └── core.py
```

```
└─ tests/
   └─ test_core.py
```

This isn't cosmetic. In a flat layout — package code sitting right next to your test scripts — your tests can accidentally import the local source files directly, because they're right there in the same directory. That works, but it's testing the wrong thing: it proves your *local files* work, not that your *installed package* works. Move the code under `src/`, and there's no accidental path to it — your tests are forced to import the version pip actually installed. That's the exact gap that lets a broken package pass all your tests and still fail for the first person who installs it for real.

Naming and versioning are permanent decisions. The `name` field is globally unique across all of PyPI — pick something already taken and your upload just fails. And once a version is published, it's published; PyPI won't let you overwrite `0.1.0` with a fix. You bump to `0.1.1` instead. This is why testing before the real publish matters so much — there's no undo button.

Dependencies close the loop. Everything your code imports that isn't the standard library goes in `dependencies`, ideally with a minimum version if your code relies on a feature from a particular release. When someone runs `pip install`, this list is what pip resolves and installs *before* your code executes — which is precisely the step that was missing when you emailed a raw `.py` file.

Python Code

```
# pyproject.toml
[build-system]
requires = ["setuptools>=68.0"]
build-backend = "setuptools.build_meta"

[project]
name = "textutils-yourname"          # must be unique on PyPI
version = "0.1.0"
description = "Small text formatting helpers"
requires-python = ">=3.9"
dependencies = [
    "python-dateutil>=2.8",
]

[tool.setuptools.packages.find]
where = ["src"]
```

```
# src/textutils/core.py
def title_case(text: str) -> str:
    """Return text with each word capitalized."""
    return " ".join(word.capitalize() for word in text.split())
```

```
# src/textutils/__init__.py
from .core import title_case

__all__ = ["title_case"]
```

Code Walkthrough

- `[build-system]` tells the build tool (`python -m build`) which backend to use to actually construct the wheel — `setuptools` here, the most common choice.
- `[project].name` is the literal string someone types into `pip install` . It has to be unique across all of PyPI, which is why real projects often suffix or prefix a name (`textutils-yourname` in the example).
- `[project].version` follows semantic-versioning-style conventions (`major.minor.patch`) and must increase on every publish — PyPI rejects re-uploads of an existing version number.
- `dependencies` is the list `pip` resolves and installs first, before your package's own code is placed. This is the line that directly fixes `ModuleNotFoundError` for anyone installing your package.
- `[tool.setuptools.packages.find]` with `where = ["src"]` tells setuptools to look inside `src/` for importable packages — this is what enforces the src layout at build time, not just by convention.
- `src/textutils/__init__.py` re-exports `title_case` so users can write `from textutils import title_case` instead of reaching into `textutils.core` .

Dry Run

Say you've written the files above and want to get `textutils` installable end to end.

1. `python -m build` — reads `pyproject.toml` , finds `src/textutils` , and produces `dist/textutils-yourname-0.1.0-py3-none-any.whl` plus a source archive. The wheel is just a zip file in a shape every Python installer knows how to unpack — no code execution required to install it.
2. `twine upload --repository testpypi dist/*` — uploads the wheel to TestPyPI, the practice index, using twine to handle authentication and the upload protocol so you don't hand-roll HTTP requests against PyPI's API.
3. In a fresh virtual environment: `pip install --index-url https://test.pypi.org/simple/ textutils-yourname` . If it installs and `from textutils import title_case` works, your contract is correct.

4. Confident it works, run `twine upload dist/*` to push the same wheel to the real PyPI index.

Now anyone, anywhere, runs `pip install textutils-yourname` and gets exactly what you tested.

Complexity Analysis

There's no algorithmic time/space complexity here — packaging isn't an algorithm, it's a build and distribution process. The relevant "cost" is operational:

- **Build time:** `python -m build` does one static pass over your source tree and metadata — proportional to project size, and effectively instant for small projects.
- **Install time:** dominated by dependency resolution and download, not your code. This is the cost you're paying to eliminate a much bigger cost — a broken install that a user has to debug themselves.
- **Irreversibility, not complexity, is the real constraint:** a taken name or a published version number is permanent. That's why TestPyPI exists — it converts an unrecoverable mistake into a recoverable one.

Alternative Solutions

Poetry and **Hatch** are alternative build backends that also read (their own extensions of) `pyproject.toml` but add their own dependency resolution, virtual environment management, and publishing commands. They're worth adopting if you want an opinionated, all-in-one workflow. For a single package with a short dependency list, plain `setuptools` plus `build` and `twine` — what this lesson covers — is simpler and has fewer moving parts, which matters more when you're learning what's actually happening under the hood.

Edge Cases

- **No dependencies at all:** `dependencies = []` or omit the key — still valid, still worth declaring `name` and `version` explicitly.
- **Duplicate package name on PyPI:** upload fails outright; you find out immediately, which is exactly why you should check availability (or just try the TestPyPI upload) before committing to a name in documentation.
- **Version already published:** re-uploading the same version number fails; you must bump the version even for a one-line fix.

- **Missing `src/__init__.py`** : without it, `setuptools.packages.find` won't discover your package, and the build silently produces an empty or broken wheel.
- **Unpinned dependency that changes behavior**: omitting a minimum version on a dependency that introduced a breaking change in a later release can make your package fail for users who happen to get a newer version pip resolves.

Common Mistakes

1. Publishing straight to real PyPI without testing on TestPyPI first — and discovering the name is taken or the wheel is broken, permanently, on the record.
2. Leaving code in a flat layout, so local tests pass against the working directory instead of the installed package, masking real install bugs.
3. Forgetting to bump the version number after a fix and having the upload rejected.
4. Missing a dependency in `pyproject.toml` because it "was already installed" in your own environment, reproducing the exact `ModuleNotFoundError` you were trying to fix.
5. Treating `requirements.txt` as the dependency source of truth instead of `pyproject.toml` , letting the two drift out of sync.
6. Picking a package name without checking availability first, then having to rename mid-project.
7. Assuming `python -m build` needs a `setup.py` — under `pyproject.toml` it doesn't, and keeping a stale one around can cause backend conflicts.

Interview Questions

- What's the difference between `pyproject.toml` and `setup.py` , and why did the ecosystem move toward the former?
- Why does a `src` layout catch import bugs that a flat layout hides?
- What's actually inside a `.whl` file, and why do installers prefer it over running arbitrary build scripts?
- Why can't you overwrite a published version on PyPI, and what workflow does that constraint force?
- What role does `twine` play that `pip` or `build` don't already handle?

Similar Problems

- **Understanding Python virtual environments** — the companion piece to packaging; isolates dependencies so `pip install` results are reproducible per project.

- **Semantic versioning in practice** — directly relevant, since PyPI's "no overwrite" rule only makes sense once you understand what major/minor/patch bumps are supposed to signal.
- **Writing a .gitignore and MANIFEST.in for Python projects** — the adjacent problem of controlling exactly what ships inside your wheel versus your source repo.

Key Takeaways

Packaging is just making two things explicit that used to be invisible: what your code needs, and where it lives relative to everything else. `pyproject.toml` is the single file that answers both. The `src` layout forces you to test what you actually ship. TestPyPI exists because PyPI's permanence — unique names, non-overwritable versions — means mistakes there are forever, so you rehearse first. Once that contract is written, `pip install your-project-name` behaves exactly like `pip install requests`, because it's the same mechanism.

Watch the Video

This lesson is easier to follow end to end on video, where every step — writing the file, running the build, testing on TestPyPI, and the real upload — happens in order. Watch it here: {LINK}

About the Series

This is part of **Fun with Learning Technology**, a series that walks through the parts of software engineering that don't show up in algorithm practice but come up constantly in real work — packaging, tooling, project structure, and the habits that separate a script from a shippable project. Each lesson builds toward being able to hand someone a working project, not just a file.

Call To Action

If this saved you from a future `ModuleNotFoundError`, the best way to support the series is simple: like the video on YouTube, subscribe so you don't miss the next one, and subscribe to the Substack for the written breakdown of every lesson. Drop a comment with what you're packaging next, and share this with anyone still emailing `.py` files around.

Quick Review

Why does a script that works locally break on another machine?

It relies on unpinned dependencies or system state that only exists on your machine — no declared, installed package list.

What does pip actually read to install a package correctly?

pyproject.toml — the standardized contract declaring metadata, dependencies, and build system for the project.

src layout vs flat layout — key difference?

src/ nests the package under a src/ folder, forcing an install to import it; flat layout imports directly from the repo root, risking accidental local imports.

Trickiest gotcha when picking a package name?

Name must be unique on PyPI and unchangeable once claimed heavily — check availability before writing code around it.

Why upload to TestPyPI before the real PyPI?

Lets you verify the built wheel installs and imports correctly without permanently consuming your real package name/version slot.

Interview follow-up: what does twine actually do vs build?

build creates the wheel/sdist artifacts; twine uploads those artifacts securely (HTTPS, auth) to PyPI/TestPyPI.